Lecture exercise branching

This notebook was generated in pseudocode mode.

Exercise: Positive, Negative, or Zero

Write a program that determines whether a given number is positive, negative, or zero.

Use the number `num = -5` and store the result in variable `result` ("positive", "negative", or "zero").

```
num = -5
# step 1: check if the number is greater than 0
    # step 2: assign "positive" to result
# step 3: otherwise, check if the number is less than 0
    # step 4: assign "negative" to result
# step 5: otherwise (the number is equal to 0)
    # step 6: assign "zero" to result
```

```
assert result == "negative"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Even or Odd Number

Write a program that determines whether a given integer is even or odd.

Use the number `num = 7` and store the result in variable `result` ("even" or "odd").

```
num = 7
# step 1: check if the number modulo 2 equals 0 (divisible by 2)
    # step 2: assign "even" to result
# step 3: otherwise (remainder is not 0)
    # step 4: assign "odd" to result
```

```
assert result == "odd"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Grade Evaluator

Write a program that converts a numerical score (0-100) to a letter grade:

- 90-100: A
- 80-89: B
- 70-79: C
- 60-69: D
- 0-59: F

Use the score `score = 85` and store the letter grade in variable `result`.

```
score = 85
# step 1: check if score is 90 or above for grade A
    # step 2: assign "A" to result
# step 3: otherwise, check if score is 80 or above for grade B
    # step 4: assign "B" to result
# step 5: otherwise, check if score is 70 or above for grade C
    # step 6: assign "C" to result
```

```
# step 7: otherwise, check if score is 60 or above for grade D
    # step 8: assign "D" to result
# step 9: otherwise (score is below 60)
    # step 10: assign "F" to result
```

```
assert result == "B"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Divisibility Checker

Write a program that checks if a given number is divisible by both 3 and 5.

Use the number `num = 30` and store the result in variable `result` ("divisible by both" or "not divisible by both").

```
num = 30
# step 1: check if number is divisible by 3 AND divisible by 5
    # step 2: use modulo operator to check if remainder is 0 for both 3 and 5
    # step 3: if both conditions are true, assign "divisible by both" to result
```

```
# step 4: otherwise
    # step 5: assign "not divisible by both" to result
```

```
assert result == "divisible by both"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Leap Year Checker

Write a program that determines whether a given year is a leap year or not.

A leap year is:

- Divisible by 4 but not by 100, OR
- Divisible by 400

Use the year `year = 2024` and store the result in variable `result` ("leap year" or "not leap year").

```
year = 2024
# step 1: check the complex leap year condition using logical operators
    # step 2: check if year is divisible by 4 AND not divisible by 100
    # step 3: OR check if year is divisible by 400
    # step 4: if either condition is true, assign "leap year" to result
# step 5: otherwise
    # step 6: assign "not leap year" to result
```

```
assert result == "leap year"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Vowel or Consonant

Write a program that determines whether a given letter is a vowel or consonant.

Use the letter `letter = "A"` and store the result in variable `result` ("vowel" or "consonant").

```
letter = "A"
# step 1: check if the letter is in the string of vowels (both lowercase and uppercase)
    # step 2: use the 'in' operator to check if letter is in "aeiouAEIOU"
    # step 3: if true, assign "vowel" to result
# step 4: otherwise
    # step 5: assign "consonant" to result
```

```
assert result == "vowel"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Password Validator

Write a program that validates a password. If the password is "password123", grant access, otherwise deny access.

Use the password `password = "password123"` and store the result in variable `result` ("Access granted" or "Access denied").

```
password = "password123"
# step 1: check if the entered password equals the correct password "password123"
    # step 2: if passwords match, assign "Access granted" to result
# step 3: otherwise
    # step 4: assign "Access denied" to result
```

```
assert result == "Access granted"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Largest of Three Numbers

Write a program that finds the largest of three given numbers.

Use the numbers `num1 = 15`, `num2 = 8`, and `num3 = 22`. Store the largest number in variable `result`.

```
num1 = 15
num2 = 8
num3 = 22
```

```
# step 1: check if num1 is greater than or equal to both num2 and num3
    # step 2: assign num1 to result as the largest
# step 3: otherwise, check if num2 is greater than or equal to both num1 and num3
    # step 4: assign num2 to result as the largest
# step 5: otherwise (num3 must be the largest)
    # step 6: assign num3 to result as the largest
```

```
assert result == 22
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Day of the Week

Write a program that converts a number (1-7) to the corresponding day of the week:

- 1: Monday
- 2: Tuesday
- 3: Wednesday
- 4: Thursday
- 5: Friday
- 6: Saturday
- 7: Sunday
- Any other number: "Invalid number"

Use the number `day = 5` and store the day name in variable `result`.

```
day = 5
# step 1: check if day equals 1
    # step 2: assign "Monday" to result
# step 3: otherwise, check if day equals 2
    # step 4: assign "Tuesday" to result
# step 5: otherwise, check if day equals 3
    # step 6: assign "Wednesday" to result
# step 7: otherwise, check if day equals 4
    # step 8: assign "Thursday" to result
# step 9: otherwise, check if day equals 5
    # step 10: assign "Friday" to result
# step 11: otherwise, check if day equals 6
    # step 12: assign "Saturday" to result
# step 13: otherwise, check if day equals 7
    # step 14: assign "Sunday" to result
```

```
# step 15: otherwise (invalid number)
    # step 16: assign "Invalid number" to result
```

```
assert result == "Friday"
```

```
# Test with invalid number
test_day = 8
if test_day == 1:
    test_result = "Monday"
elif test_day == 2:
    test_result = "Tuesday"
elif test_day == 3:
    test_result = "Wednesday"
elif test_day == 4:
    test_result = "Thursday"
elif test_day == 5:
    test_result = "Friday"
elif test_day == 6:
    test_result = "Saturday"
elif test_day == 7:
    test_result = "Sunday"
else:
    test_result = "Invalid number"
assert test_result == "Invalid number"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Simple Calculator

Write a program that performs basic arithmetic operations (+, -, *, /) on two numbers based on the given operation.

Use `num1 = 10.0`, `num2 = 3.0`, and `operation = "+"`. Store the result in variable `result`. Handle division by zero with an error message.

```
num1 = 10.0
num2 = 3.0
operation = "+"
# step 1: check if operation is addition ("+")
    # step 2: perform addition and assign to result
# step 3: otherwise, check if operation is subtraction ("-")
    # step 4: perform subtraction and assign to result
# step 5: otherwise, check if operation is multiplication ("*")
    # step 6: perform multiplication and assign to result
# step 7: otherwise, check if operation is division ("/")
    # step 8: check if second number is not zero to avoid division by zero
        # step 9: perform division and assign to result
    # step 10: otherwise (if dividing by zero)
        # step 11: assign error message to result
```

```
# step 12: otherwise (invalid operation)
    # step 13: assign error message to result
```

```
assert result == 13.0
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Triangle Type Classifier

Write a program that determines the type of triangle based on the lengths of its three sides:

- Equilateral: All sides are equal
- Isosceles: Two sides are equal
- Scalene: No sides are equal

Use `side1 = 5.0`, `side2 = 5.0`, and `side3 = 8.0`. Store the triangle type in variable `result`.

```
side1 = 5.0
side2 = 5.0
```

```
side3 = 8.0
# step 1: check if all three sides are equal (equilateral triangle)
    # step 2: compare side1 == side2 == side3
    # step 3: if true, assign "equilateral" to result
# step 4: otherwise, check if any two sides are equal (isosceles triangle)
    # step 5: check if side1 == side2 OR side2 == side3 OR side1 == side3
    # step 6: if true, assign "isosceles" to result
# step 7: otherwise (no sides are equal)
    # step 8: assign "scalene" to result
```

```
assert result == "isosceles"
```

Python Tutor Visualization

The following cells contain code to generate links to Python Tutor for visualizing this exercise. You need to first run the cell containing your solution code before running the cell containing the Python Tutor link.

```
from IPython.display import HTML;import urllib.parse as u;HTML(f'<a href="https://pythontutor
```

Exercise: Maximum of Three Numbers

Write a program that finds the maximum (largest) of three given numbers.

Use the numbers `num1 = 12.5`, `num2 = 8.3`, and `num3 = 15.7`. Store the maximum number in variable `result`.

```
num1 = 12.5
num2 = 8.3
num3 = 15.7
# step 1: check if num1 is greater than or equal to both num2 and num3
    # step 2: assign num1 to result as the maximum
# step 3: otherwise, check if num2 is greater than or equal to both num1 and num3
    # step 4: assign num2 to result as the maximum
# step 5: otherwise (num3 must be the maximum)
    # step 6: assign num3 to result as the maximum
```

```
assert result == 15.7
```

```
# Test with different maximum
test_num1, test_num2, test_num3 = 25.0, 18.5, 20.2
if test_num1 >= test_num2 and test_num1 >= test_num3:
    test_result = test_num1
elif test_num2 >= test_num1 and test_num2 >= test_num3:
    test_result = test_num2
else:
    test_result = test_num3
assert test_result == 25.0
```